南方科技大学
SOUTHERN UNIVERSITY OF SCIENCE AND TECHNOLOGY

计算机组成原理
COMPUTER ORGANIZATION

本课件内容参考自David A. Patterson与John L. Hennessy的教材《Computer Organization and Design: The Hardware/Software Interface》官方配套课件。

zhangj4@sustech.edu.cn

Computer Organization

Lecture 12

Cache Optimization

zhangj4@sustech.edu.cn

Outline

- Measuring and Improving Cache Performance
- Multilevel Cache Performance
- Hamming Error-Correcting Code

zhangj4@sustech.edu.cn

Measuring and Improving Cache Performance

zhangj4@sustech.edu.cn

Measuring Cache Performance

- Components of CPU time
  - Program execution cycles
    - Includes cache hit time
  - Memory stall cycles
    - Mainly from cache misses
- With simplifying assumptions:

$$\text{Memory stall cycles} = \frac{\text{Memory accesses}}{\text{Program}} \times \text{Miss rate} \times \text{Miss penalty}$$

$$= \frac{\text{Instructions}}{\text{Program}} \times \frac{\text{Misses}}{\text{Instruction}} \times \text{Miss penalty}$$

Cache Performance Example

- Given
  - I-cache miss rate = 2%
  - D-cache miss rate = 4%
  - Miss penalty = 100 cycles
  - Base CPI (ideal cache) = 2
  - Load & stores are 36% of instructions
- Miss cycles per instruction
  - I-cache: 100% × 2% × 100 = 2
  - D-cache: 36% × 4% × 100 = 1.44
- Effective CPI = 2 + 2 + 1.44 = 5.44
  - Ideal CPU is 5.44/2 =2.72 times faster

Average Access Time

- Hit time is also important for performance
- Average memory access time (**AMAT**)

$$\text{AMAT} = \text{Hit time} + \text{Miss rate} \times \text{Miss penalty}$$

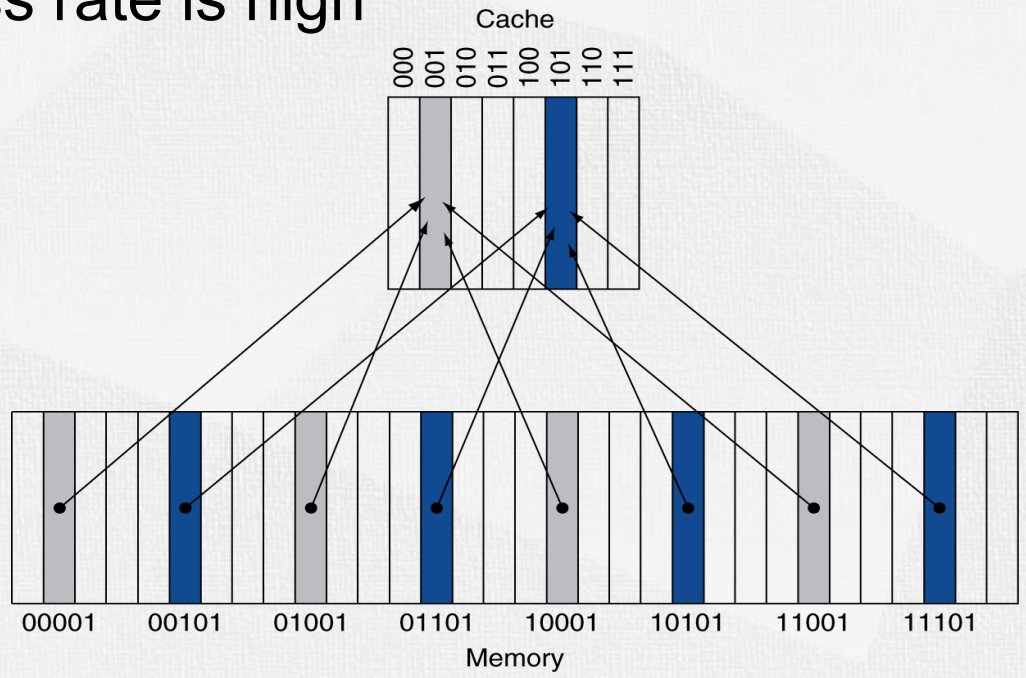
- Example
  - CPU with 1ns clock, hit time = 1 cycle, miss penalty = 20 cycles, I-cache miss rate = 5%
  - AMAT = 1 + 0.05 × 20 = 2cycles = 2ns
    - 2 cycles per instruction

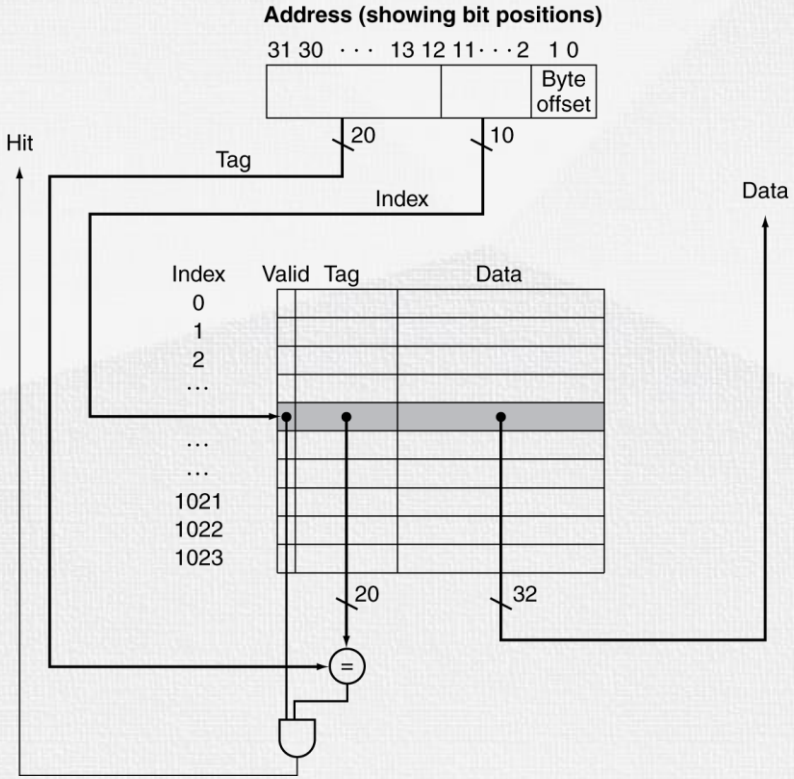
Performance Summary

- When CPU performance increased
  - Miss penalty becomes more significant
- Decreasing base CPI
  - Greater proportion of time spent on memory stalls
- Increasing clock rate
  - Memory stalls account for more CPU cycles
- Can’t neglect cache behavior when evaluating system performance

Recall: Direct Mapped Cache

- Direct mapped cache:
  - Location determined by address
  - One data in memory is mapped to **only one location** in cache
  - Capacity of cache is not fully exploited
  - Miss rate is high





zhangj4@sustech.edu.cn

Recall: Direct Mapped Cache

16	10 000	Miss	000
3	00 011	Miss	011
16	10 000	Hit	000

Index	V	Tag	Data
000	**Y**	**10**	**Mem[10000]**
001	N		
010	Y	11	Mem[11010]
011	**Y**	**00**	**Mem[00011]**
100	N		
101	N		
110	Y	10	Mem[10110]
111	N		

18	10 010	Miss	010
16	10 000	Hit	000

Index	V	Tag	Data
000	Y	10	Mem[10000]
001	N		
010	**Y**	**10**	**Mem[10010]**
011	Y	00	Mem[00011]
100	N		
101	N		
110	Y	10	Mem[10110]
111	N		

zhangj4@sustech.edu.cn

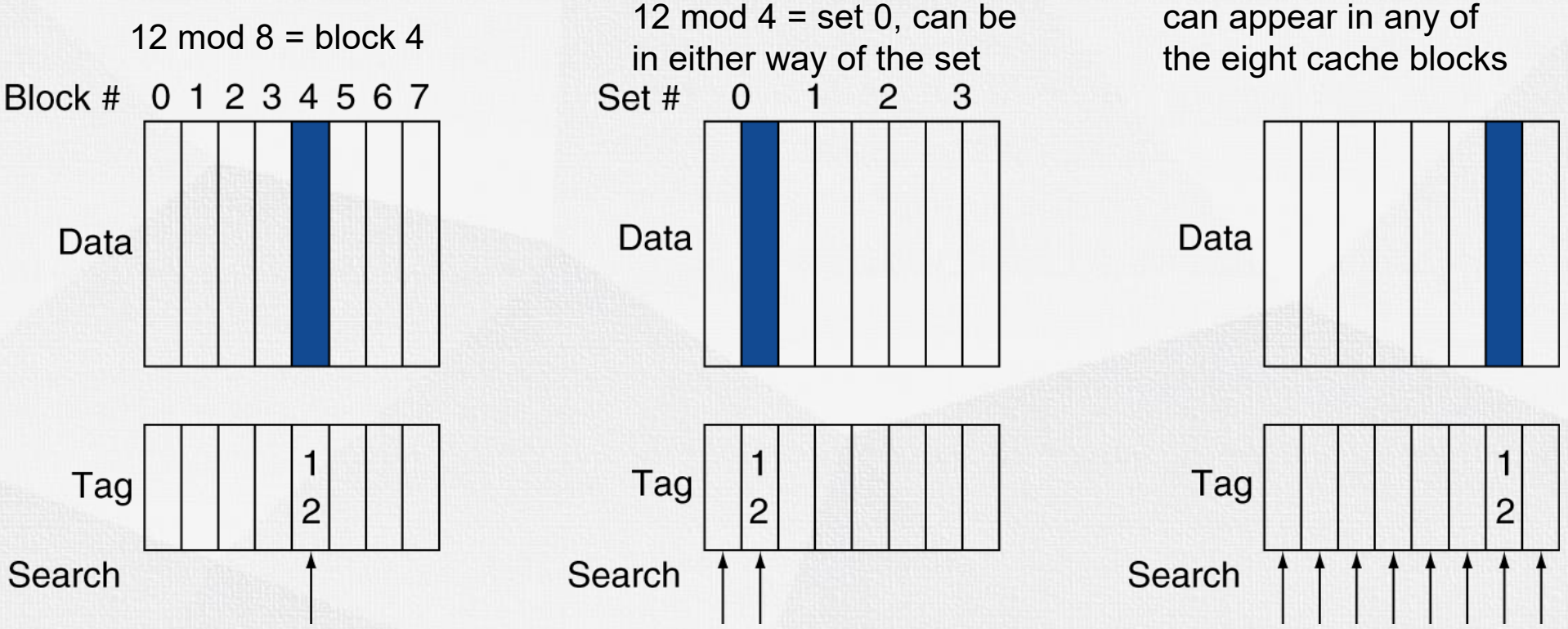
Associative Caches

- Fully associative
  - Allow a given block to go in any cache entry
  - Requires all entries to be searched at once
  - Comparator per entry (expensive)
- n-way set associative
  - Each set contains *n* entries
  - Block number determines which set
    - (Block number) modulo (#Sets in cache)
  - Search all entries in a given set at once
  - n comparators (less expensive)

zhangj4@sustech.edu.cn

Associative Cache Example

- Example: Placement of a block whose address is 12:



Direct mapped

suppose 8 block cache
# of block = 8

Set associative

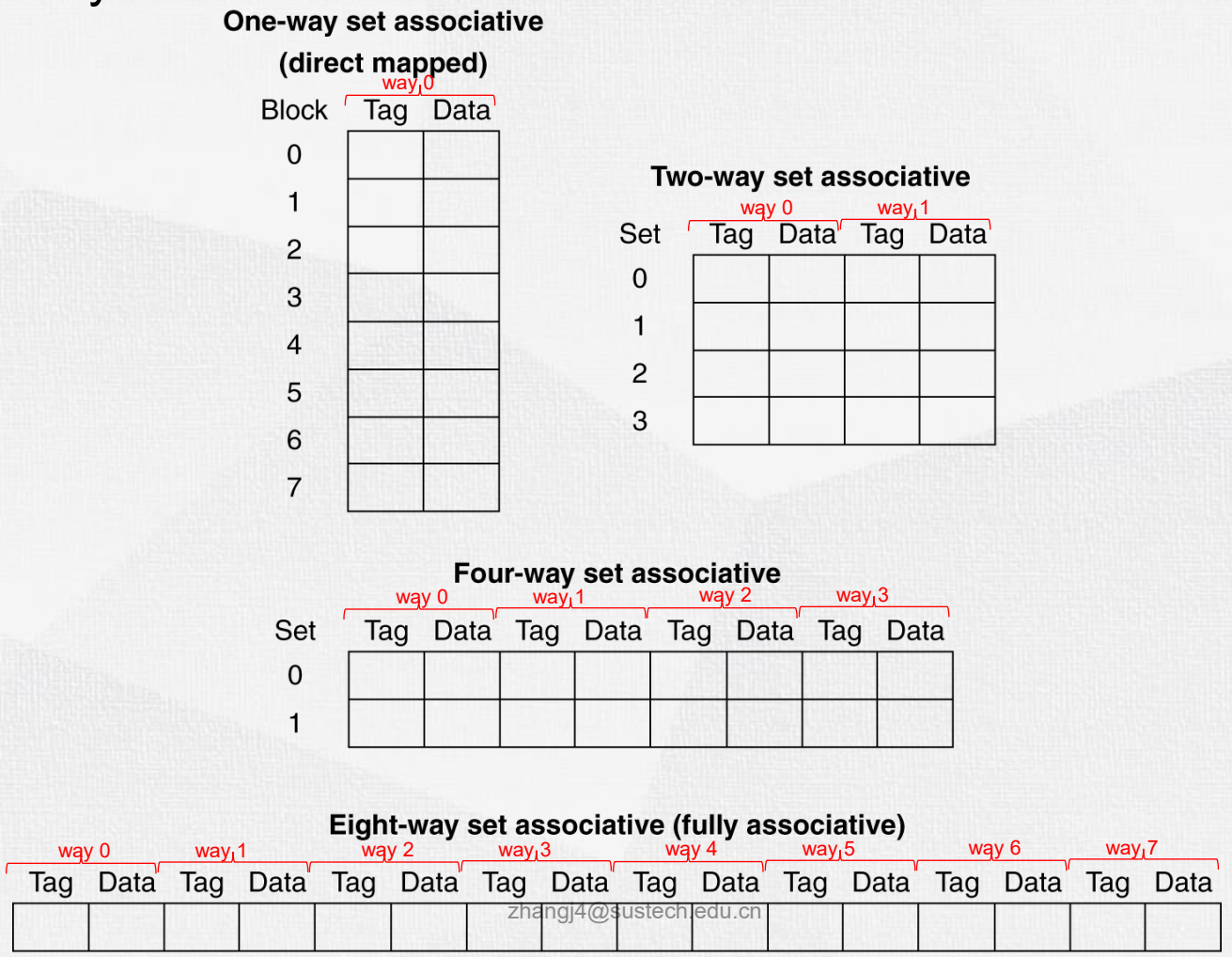
suppose 8 block 2-way associative cache
# of set = 4

Fully associative

suppose 8 block fully associative cache
no index

Spectrum of Associativity

- An eight-block cache configured as direct mapped, two-way set associative, four-way set associative, and fully associative.



Associativity Example

- 4-block caches, 1byte/block
  - Direct mapped, 2-way set associative, fully associative
  - Block access sequence: 0, 8, 0, 6, 8
- Direct mapped
  - index = (Block address) mod (**#Blocks**)

needs 2 bit for index

Block address	Cache index	Hit/miss	Cache content after access			
			way 0	way 0	way 1	way 1
			block 0	block 1	block 2	block 3
0	0	miss	**Mem[0]**			
8	0	miss	**Mem[8]**			
0	0	miss	**Mem[0]**			
6	2	miss	Mem[0]		**Mem[6]**	
8	0	miss	**Mem[8]**		Mem[6]	

Direct Mapped Cache Reference Detail

addr	block index	tag	hit/miss
0	00	00	Compulsory Miss
8	00	10	Conflict Miss
0	00	00	Conflict Miss
6	10	01	Compulsory Miss
8	00	10	Conflict Miss

	Tag	Data
0	00	Mem[0]
1		
2		
3		

address 0

	Tag	Data
0	10	Mem[8]
1		
2		
3		

address 8

	Tag	Data
0	00	Mem[0]
1		
2		
3		

address 0

	Tag	Data
0	00	Mem[0]
1		
2	01	Mem[6]
3		

address 6

	Tag	Data
0	10	Mem[8]
1		
2	01	Mem[6]
3		

address 8

zhangj4@sustech.edu.cn

Associativity Example (cont.)

Block access sequence: 0, 8, 0, 6, 8

- 2-way set associative
  - index = (Block address) mod (**#Sets**)

needs 1 bit for index

Block address	Cache index	Hit/miss	Cache content after access			
			Set 0		Set 1	
			way 0	way 1	way 0	way 1
0	0	miss	**Mem[0]**			
8	0	miss	Mem[0]	**Mem[8]**		
0	0	hit	**Mem[0]**	Mem[8]		
6	0	miss	Mem[0]	**Mem[6]**		
8	0	miss	**Mem[8]**	Mem[6]		

2-Way Set Associative Cache Reference Detail

addr	set index	tag	hit/miss
0	0	000	Compulsory Miss
8	0	100	Compulsory Miss
0	0	000	Hit
6	0	011	Conflict Miss
8	0	100	Conflict Miss

	Tag	Data	Tag	Data
0	000	Mem[0]		
1				

address 0

	Tag	Data	Tag	Data
0	000	Mem[0]	100	Mem[8]
1				

address 8

	Tag	Data	Tag	Data
0	00	Mem[0]	100	Mem[8]
1				

address 0

	Tag	Data	Tag	Data
0	000	Mem[0]	011	Mem[6]
1				

address 6

	Tag	Data	Tag	
0	100	Mem[8]	011	Mem[6]
1				

address 8

zhangj4@sustech.edu.cn

Associativity Example (cont.)

- Fully associative (no index)

Block access sequence: 0, 8, 0, 6, 8

needs 0 bit for index

Block address		Hit/miss	Cache content after access			
			way 0	way 1	way 2	way 3
0		miss	**Mem[0]**			
8		miss	Mem[0]	**Mem[8]**		
0		hit	**Mem[0]**	Mem[8]		
6		miss	Mem[0]	Mem[8]	**Mem[6]**	
8		hit	Mem[0]	**Mem[8]**	Mem[6]	

Fully Associative Cache Reference Detail

addr	tag	hit/miss
0	000	Compulsory Miss
8	100	Compulsory Miss
0	000	Hit
6	011	Compulsory Miss
8	100	Hit

address 0

Tag	Data	Tag	Data	Tag	Data	Tag	Data
0000	Mem[0]						

address 8

Tag	Data	Tag	Data	Tag	Data	Tag	Data
0000	Mem[0]	1000	Mem[8]				

address 0

Tag	Data	Tag	Data	Tag	Data	Tag	Data
0000	Mem[0]	1000	Mem[8]				

address 6

Tag	Data	Tag	Data	Tag	Data	Tag	Data
0000	Mem[0]	1000	Mem[8]	0110	Mem[6]		

address 8

Tag	Data	Tag	Data	Tag	Data	Tag	Data
0000	Mem[0]	1000	Mem[8]	0110	Mem[6]		

zhangj4@sustech.edu.cn

How Much Associativity

- Increased associativity decreases miss rate
  - But with diminishing returns
- Miss rate simulation of a system with 64KB D-cache, 16-word blocks, SPEC2000
  - 1-way: 10.3%
  - 2-way: 8.6%
  - 4-way: 8.3%
  - 8-way: 8.1%

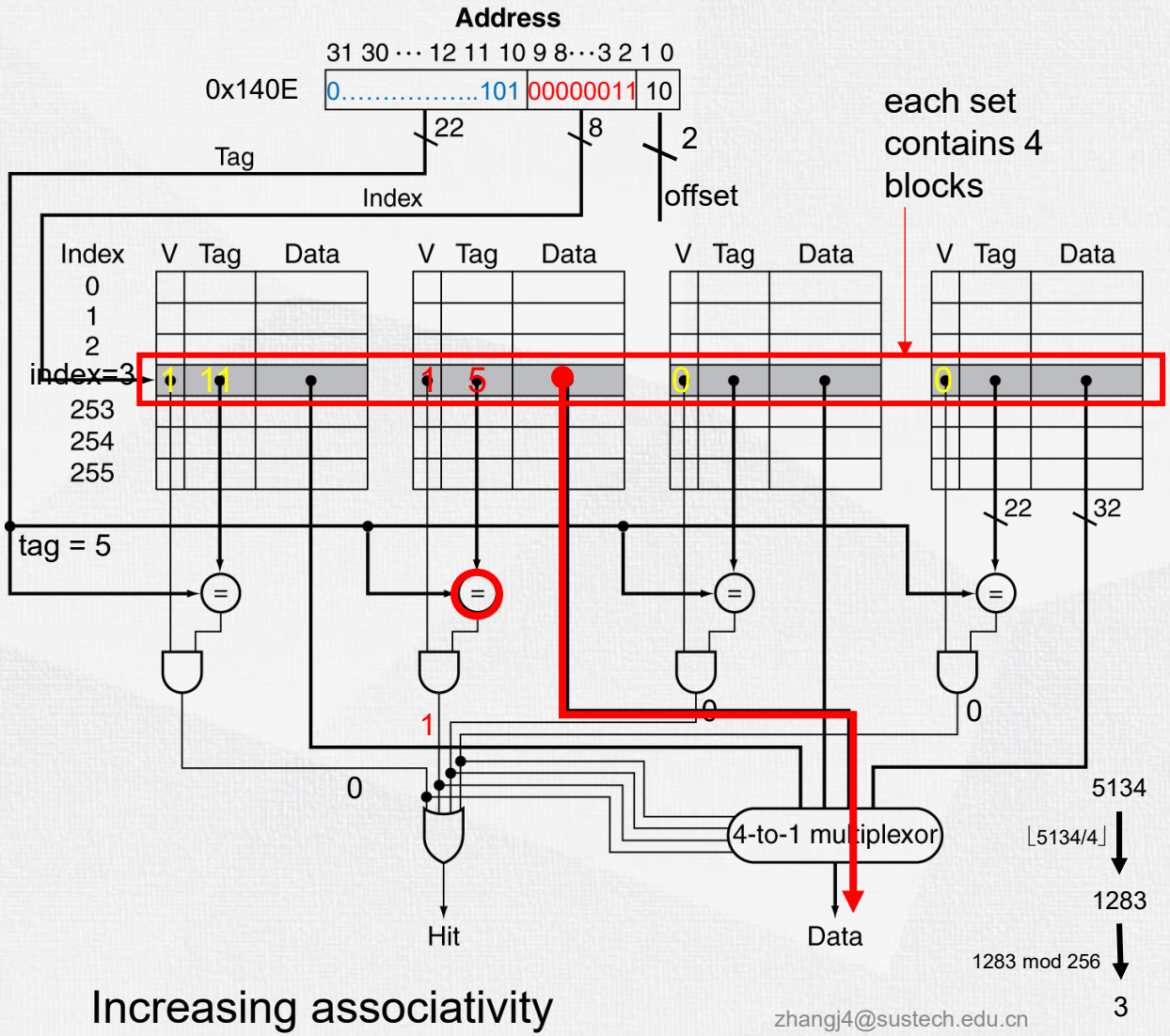
Address Subdivision

- 4K blocks cache, 4-word/block, 32-bit address
- offset = $\log_2 16$ = 4bits → 28bits for index + tag for all types
- direct mapped (1-way set associative)
  - same number of sets as blocks →4096 blocks(sets)
  - index = $\log_2 4096$ = 12 bits
  - tag = 32 – 4 – 12 = 16 bits
- 2-way set associative
  - 2 blocks/set → 2048 sets
  - index = $\log_2 2048$= 11 bits
  - tag = 32 – 4 – 11 = 17 bits
- 4-way set associative
  - 4 blocks/set → 1024 sets
  - index = $\log_2 1024$= 10 bits
  - tag = 32 – 4 – 10 = 18 bits
- fully associative
  - just 1 set → no index
  - tag = 32 – 4 = 28

Q: for each struction
- # of blocks/sets
- index bit width
- tag bit width

zhangj4@sustech.edu.cn

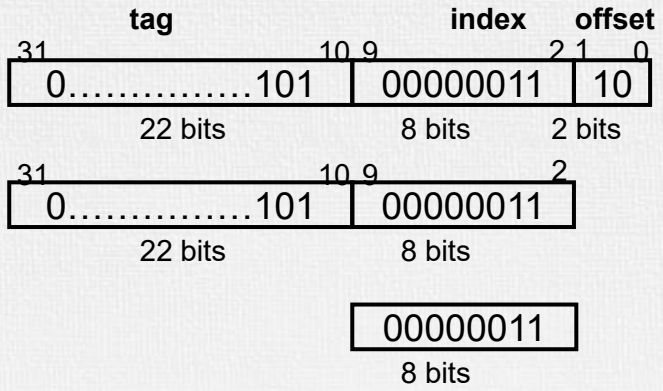
Set Associative Cache Organization



4KB cache, 1word/block
To what **set** number does address 0x140E map?

offset: $\log_2$(4byte) = 2bits
#blocks: 4KB/4B = 1K blocks
#sets: 1K blocks/4 way = 256 sets
index: $\log_2$(256sets) = 8bits

$0x140E_{hex}$=1010000001110_{two}
tag index offset



Increasing associativity shrinks index, expands tag

zhangj4@sustech.edu.cn

Replacement Policy

- Direct mapped: no choice
- Set associative
  - Prefer non-valid entry, if there is one
  - Otherwise, choose among entries in the set
- Least-recently used (LRU)
  - Choose the one unused for the longest time
    - Simple for 2-way, manageable for 4-way, too hard beyond that
- Random
  - Gives approximately the same performance as LRU for high associativity

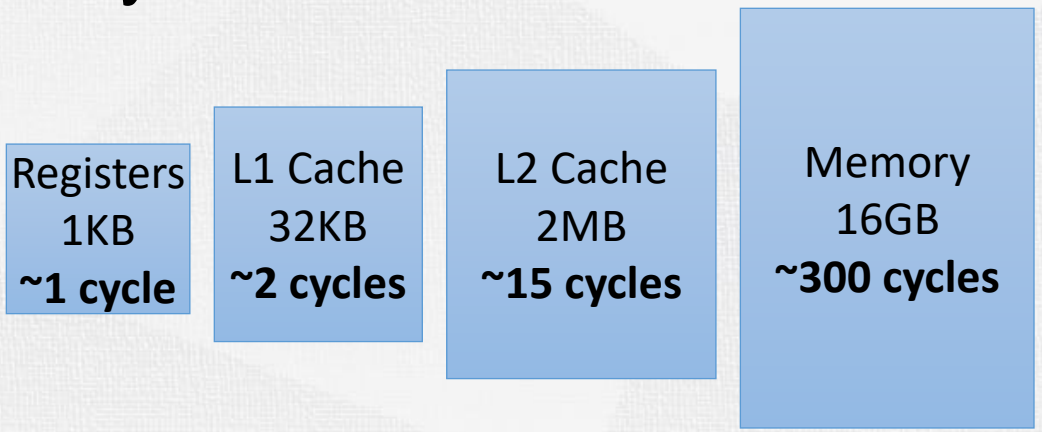
zhangj4@sustech.edu.cn

Multilevel Cache Performance

zhangj4@sustech.edu.cn

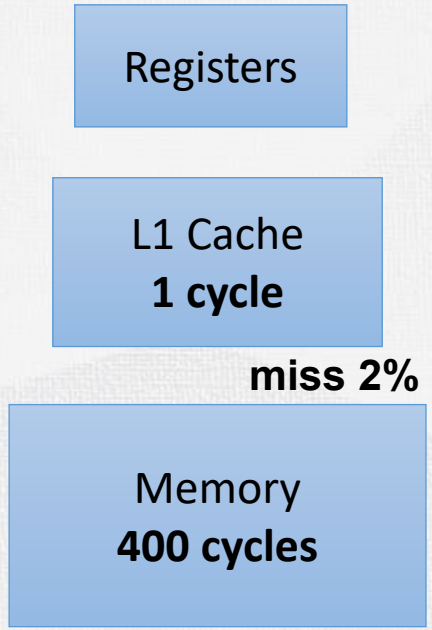
Multilevel Caches

- Primary cache attached to CPU
  - Small, but fast
- Level-2 cache services misses from primary cache
  - Larger, slower, but still faster than main memory
- Main memory services L-2 cache misses
- Some high-end systems include L-3 cache



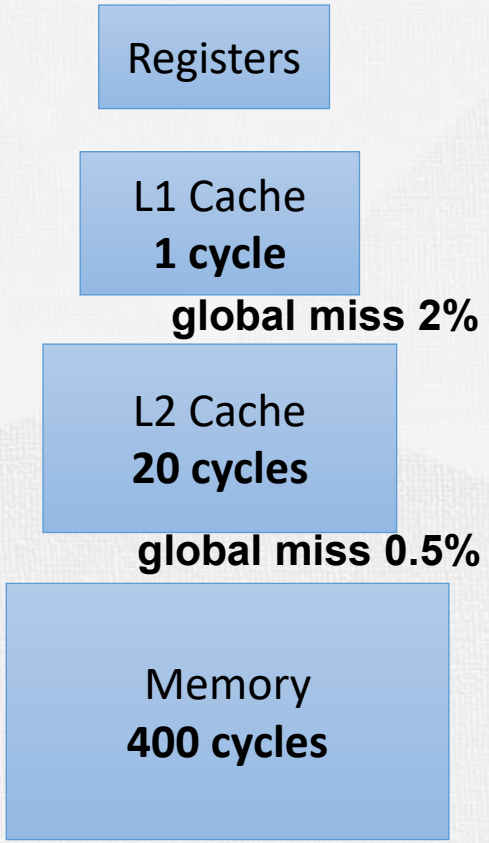
Multilevel Cache Example

- Given
  - CPU base CPI = 1, clock rate = 4GHz
  - Primary cache Miss rate/instruction = 2%
  - Main memory access time = 100ns
- Solution: With just primary cache
  - Miss penalty = 100ns/0.25ns = 400 cycles
  - Effective CPI = Base CPI + miss penalty/instruction
    = 1 + 2% × 400 = 9



Example (cont.)

- Given: After adding L-2 cache
  - L2 cache Access time = 5ns
  - L2 global miss rate/instruction = 0.5%
- Solution 1, calculate based on Global miss rate (The fraction of references that miss in all levels):
  - L-1 miss (2%) first need to access the L2 cache
    - Penalty = 5ns/0.25ns = 20 cycles
  - L-1 miss with L-2 also miss (0.5%)
    - Extra penalty = 400 cycles
  - Effective CPI = 1 + 2%×20 + 0.5%×400 = 3.4
  - Speedup = 9/3.4 = 2.6
- Solution 2, calculate based on Local miss rate (The fraction of references to one level of a cache that miss)
  - L2 local miss rate/instruction = 0.5%/2% = 25%
  - Effect CPI = 1 + 2%×(20 + 25%×400) = 3.4



Multilevel Cache Considerations

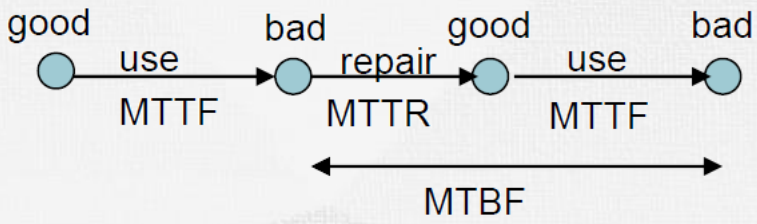
- Primary cache
  - Focus on minimal hit time
- L-2 cache
  - Focus on low miss rate to avoid main memory access
  - Hit time has less overall impact
- Results
  - L-1 cache usually smaller than a single cache
  - L-1 block size smaller than L-2 block size

Hamming Error-Correcting Code

zhangj4@sustech.edu.cn

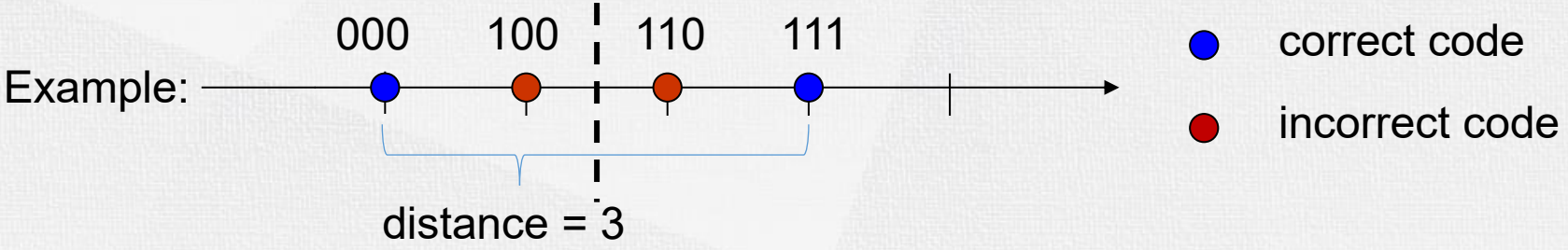
Dependability Measures

- Reliability: mean time to failure (MTTF)
  - e.g. MTTF of some disks is around 1,000,000-hour (114 years)
  - e.g. MTTF of fan is around 70,000-hour (8 years)
- Mean time to repair (MTTR)
  - the average time required to repair a failed system
- Mean time between failures
  - MTBF = MTTF + MTTR
- Availability = MTTF /(MTTF +MTTR)
  - increase MTTF: fault avoidance, fault tolerance, fault forecasting
  - reduce MTTR: fault detection, fault diagnosis and fault repair
- “nines of availability” per year
  - One nine: 90% → 36.5 days of repair/year (2.4h/day)
  - Two nines: 99% → 3.65 days of repair/year (14.1 min/day)
  - Thee nines: 99.9% → 526 minutes of repair/year (1.4 min/day)



Hamming Code

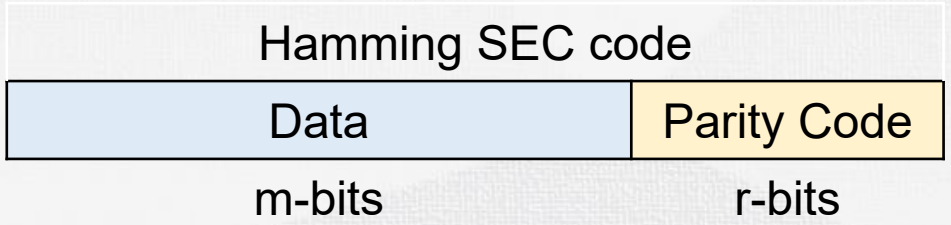
- Hamming distance
  - minimum number of bits that are different between any two correct bit patterns
  - E.g.use 111 to represent 1, use 000 to represent 0, hamming distance(d) is 3, d = 3
- Minimum distance = 2 provides single bit error detection
  - E.g. parity code: 10→101, 11→110, d = 2
- Minimum distance = 3 provides **Single Error Correction** (SEC)



Hamming Code

- To calculate how many bits are needed for Hamming SEC, let *p* be total number of parity bits and *d* number of data bits in *m* + *r* bit word. If *r* error correction bits are to point to error bit (*m* + *r* cases) plus one case to indicate that no error exists, we need:

$$2^r >= m + r + 1 \text{ bits}$$



- Example:
  - for 8 bits data, we need 4 parity bits
  - for 64 bits data, we need 7 parity bits

zhangj4@sustech.edu.cn

Hamming SEC Encoding

- To calculate the sequence 1001_1010 (0x9A) 's Hamming code:
  - $2^r >= m + r + 1$ bits → r = 4 (4 parity bits)
  - Start numbering bits from 1 on the left, All bit positions that are a power of 2 are parity bits (e.g. bit 1,2,4,8, … are parity bits), the rest are for data bits.
  - Set parity bits to create even parity for each group.

Bit position		1	2	3	4	5	6	7	8	9	10	11	12
		p1	p2	1	p4	0	0	1	p8	1	0	1	0
Encoded date bits		p1	p2	d1	p4	d2	d3	d4	p8	d5	d6	d7	d8
Parity bit coverate	p1	X		X		X		X		X		X	
	p2		X	X			X	X			X	X	
	p4				X	X	X	X					X
	p8								X	X	X	X	X

zhangj4@sustech.edu.cn

Hamming SEC Encoding

- The position of parity bit determines sequence of data bits that it checks:
  - p1 = d1 ⊕ d2 ⊕ d4 ⊕ d5 ⊕ d7

Bit position		1	2	3	4	5	6	7	8	9	10	11	12
		0		1		0	0	1		1	0	1	0
Encoded date bits		p1	p2	d1	p4	d2	d3	d4	p8	d5	d6	d7	d8
Parity bit coverate	p1	X		X		X		X		X		X	
	p2		X	X			X	X			X	X	
	p4				X	X	X	X					X
	p8								X	X	X	X	X

zhangj4@sustech.edu.cn

Hamming SEC Encoding

- The position of parity bit determines sequence of data bits that it checks:
  - p1 = d1 ⊕ d2 ⊕ d4 ⊕ d5 ⊕ d7
  - p2 = d1 ⊕ d3 ⊕ d4 ⊕ d6 ⊕ d7

Bit position		1	2	3	4	5	6	7	8	9	10	11	12
		0	1	1		0	0	1		1	0	1	0
Encoded date bits		p1	p2	d1	p4	d2	d3	d4	p8	d5	d6	d7	d8
Parity bit coverate	p1	X		X		X		X		X		X	
	p2		X	X			X	X			X	X	
	p4				X	X	X	X					X
	p8								X	X	X	X	X

zhangj4@sustech.edu.cn

Hamming SEC Encoding

- The position of parity bit determines sequence of data bits that it checks:
  - p1 = d1 ⊕ d2 ⊕ d4 ⊕ d5 ⊕ d7
  - p2 = d1 ⊕ d3 ⊕ d4 ⊕ d6 ⊕ d7
  - p4 = d2 ⊕ d3 ⊕ d4 ⊕ d8

Bit position		1	2	3	4	5	6	7	8	9	10	11	12
		0	1	1	1	0	0	1		1	0	1	0
Encoded date bits		p1	p2	d1	p4	d2	d3	d4	p8	d5	d6	d7	d8
Parity bit coverate	p1	X		X		X		X		X		X	
	p2		X	X			X	X			X	X	
	p4				X	X	X	X					X
	p8								X	X	X	X	X

zhangj4@sustech.edu.cn

Hamming SEC Encoding

- The position of parity bit determines sequence of data bits that it checks:
  - p1 = d1 ⊕ d2 ⊕ d4 ⊕ d5 ⊕ d7
  - p2 = d1 ⊕ d3 ⊕ d4 ⊕ d6 ⊕ d7
  - p4 = d2 ⊕ d3 ⊕ d4 ⊕ d8
  - p8 = d5 ⊕ d6 ⊕ d7 ⊕ d8

Bit position		1	2	3	4	5	6	7	8	9	10	11	12
		0	**1**	1	**1**	0	0	1	**0**	1	0	1	0
Encoded date bits		p1	p2	d1	p4	d2	d3	d4	p8	d5	d6	d7	d8
Parity bit coverate	p1	X		X		X		X		X		X	
	p2		X	X			X	X			X	X	
	p4				X	X	X	X					X
	p8								**X**	**X**	**X**	**X**	**X**

zhangj4@sustech.edu.cn

Hamming SEC Encoding

- The position of parity bit determines sequence of data bits that it checks:
  - p1 = d1 ⊕ d2 ⊕ d4 ⊕ d5 ⊕ d7
  - p2 = d1 ⊕ d3 ⊕ d4 ⊕ d6 ⊕ d7
  - p4 = d2 ⊕ d3 ⊕ d4 ⊕ d8
  - p8 = d5 ⊕ d6 ⊕ d7 ⊕ d8

Hamming SEC code = 0111_0010_1010 (0x72A)

Bit position		1	2	3	4	5	6	7	8	9	10	11	12
		0	**1**	1	**1**	0	0	1	**0**	1	0	1	0
Encoded date bits		p1	p2	d1	p4	d2	d3	d4	p8	d5	d6	d7	d8
Parity bit coverate	p1	X		X		X		X		X		X	
	p2		X	X			X	X			X	X	
	p4				X	X	X	X					X
	p8								X	X	X	X	X

zhangj4@sustech.edu.cn

SEC Decoding – Error Correction

- Detect error in hamming sequence 0111_0010_1110 (0x72E, 1 bit flipped)
  - The correctness of parity bits indicates which bits are in error
    - c1 = p1 ⊕ d1 ⊕ d2 ⊕ d4 ⊕ d5 ⊕ d7

Bit position		1	2	3	4	5	6	7	8	9	10	11	12
		0	1	1	1	0	0	1	0	1	1	1	0
Encoded date bits		p1	p2	d1	p4	d2	d3	d4	p8	d5	d6	d7	d8
Parity bit coverate	p1	X		X		X		X		X		X	
	p2		X	X			X	X			X	X	
	p4				X	X	X	X					X
	p8								X	X	X	X	X

c1=0 √

zhangj4@sustech.edu.cn

SEC Decoding – Error Correction

- Detect error in hamming sequence 0111_0010_1110 (0x72E, 1 bit flipped)
  - The correctness of parity bits indicates which bits are in error
    - c1 = p1 ⊕ d1 ⊕ d2 ⊕ d4 ⊕ d5 ⊕ d7
    - c2 = p2 ⊕ d1 ⊕ d3 ⊕ d4 ⊕ d6 ⊕ d7

Bit position		1	2	3	4	5	6	7	8	9	10	11	12
		0	1	1	1	0	0	1	0	1	1	1	0
Encoded date bits		p1	p2	d1	p4	d2	d3	d4	p8	d5	d6	d7	d8
Parity bit coverate	p1	X		X		X		X		X		X	
	p2		X	X			X	X			X	X	
	p4				X	X	X	X					X
	p8								X	X	X	X	X

c1=0 √

c2=1 ×

zhangj4@sustech.edu.cn

SEC Decoding – Error Correction

- Detect error in hamming sequence 0111_0010_1110 (0x72E, 1 bit flipped)
  - The correctness of parity bits indicates which bits are in error
    - c1 = p1 ⊕ d1 ⊕ d2 ⊕ d4 ⊕ d5 ⊕ d7
    - c2 = p2 ⊕ d1 ⊕ d3 ⊕ d4 ⊕ d6 ⊕ d7
    - c4 = p4 ⊕ d2 ⊕ d3 ⊕ d4 ⊕ d8

Bit position		1	2	3	4	5	6	7	8	9	10	11	12
		0	1	1	1	0	0	1	0	1	1	1	0
Encoded date bits		p1	p2	d1	p4	d2	d3	d4	p8	d5	d6	d7	d8
Parity bit coverate	p1	X		X		X		X		X		X	
	p2		X	X			X	X			X	X	
	p4				X	X	X	X					X
	p8								X	X	X	X	X

c1=0 √

c2=1 ×

c4=0 √

zhangj4@sustech.edu.cn

SEC Decoding – Error Correction

- Detect error in hamming sequence 0111_0010_1110 (0x72E, 1 bit flipped)
  - The correctness of parity bits indicates which bits are in error
    - c1 = p1 ⊕ d1 ⊕ d2 ⊕ d4 ⊕ d5 ⊕ d7
    - c2 = p2 ⊕ d1 ⊕ d3 ⊕ d4 ⊕ d6 ⊕ d7
    - c4 = p4 ⊕ d2 ⊕ d3 ⊕ d4 ⊕ d8
    - c8 = p8 ⊕ d5 ⊕ d6 ⊕ d7 ⊕ d8

Bit position		1	2	3	4	5	6	7	8	9	10	11	12
		0	1	1	1	0	0	1	0	1	1	1	0
Encoded date bits		p1	p2	d1	p4	d2	d3	d4	p8	d5	d6	d7	d8
Parity bit coverate	p1	X		X		X		X		X		X	
	p2		X	X			X	X			X	X	
	p4				X	X	X	X					X
	p8								X	X	X	X	X

c1=0 √

c2=1 ×

c4=0 √

c8=1 ×

zhangj4@sustech.edu.cn

SEC Decoding – Error Correction

- Detect error in hamming sequence 0111_0010_1110 (0x72E, 1 bit flipped)
  - The correctness of parity bits indicates which bits are in error
  - Parity bits checking result: $C_8C_4C_2C_1$=1010, indicates bit position 1010_{bin}(10) was flipped
  - Corrected hamming sequence: 0111_0010_1010 (0x72A)
  - **Original data: 1001_1010 (0x9A)**

Bit position		1	2	3	4	5	6	7	8	9	10	11	12
		0	1	1	1	0	0	1	0	1	1→0	1	0
Encoded date bits		p1	p2	d1	p4	d2	d3	d4	p8	d5	d6	d7	d8
Parity bit coverate	p1	X		X		X		X		X		X	
	p2		X	X			X	X			X	X	
	p4				X	X	X	X					X
	p8								X	X	X	X	X

c1=0 √
c2=1 ×
c4=0 √
c8=1 ×

SEC/DED Code

- Make Hamming distance = 4, single error correction (SEC), 2 bit / double error detection (DED)
- Add an additional parity bit for the whole word (p_0) to check the hamming code's parity.
- Decoding, let C = ECC groups parity check ($C_8C_4C_2C_1$), H = whole word parity check
  - C = 0(correct), H = 0(correct) → no error
  - C ≠ 0(incorrect), H ≠ 0(incorrect) → correctable single bit error
  - C = 0(correct), H ≠ 0(incorrect) → error in p_0 bit
  - C ≠ 0(incorrect), H = 0(correct) → double error occurred
- ECC DRAM uses SEC/DED with 8 bits protecting each 64 bits

zhangj4@sustech.edu.cn